UNIVERSIDAD EVANGÉLICA DE EL SALVADOR

FACULTAD DE INGENIERÍA



INGENIERÍA EN DESARROLLO DE SOFTWARE Y CIENCIA DE DATOS

Asignatura:

Programación II

Catedrático:

Ing. Daniel Enrique Guevara Gómez

AVANCE N°3 DEL PROYECTO DE INVESTIGACIÓN

GRUPO N°1

Fecha de entrega:

25 de Octubre del 2026

Integrantes:

1. **2026010077 – Díaz Escobar, Ivan Fernando (Participó)**



2. **2026010554 – Serrano Alfaro, Miguel Ángel (Participó)**



3. **2026010182 – Ruiz Fernández, Katya Abigail (Participó)**



4. **2026010255 – Pinto Elizondo, Adrian Enrique (Participó)**



5. **2026010232 – Emestica Ramírez, Alexander Alberto (Participó)**



6. **2026011324 – Flamenco Joya, Jairo Joaquín (Participó)**



7. **2026010465 – Ruiz Estrada, Vladimir Antonio (Participó)**



8. **2026010101 – Mejia Rivera, Brandon Ismael (Participó)**



1. Descripción del Problema.

Las empresas comercializadoras enfrentan frecuentemente la desconexión entre su canal de ventas digital y su control interno de almacén. Esta falta de sincronización provoca problemas críticos como la sobreventa de productos sin existencias reales, inconsistencias entre el kardex físico y los pedidos facturados, cuellos de botella al auditar catálogos extensos y vulnerabilidades en el manejo de transacciones de pago. Ante esta problemática, surge la necesidad de construir un sistema monolítico modular que centralice en tiempo real el catálogo comercial y las existencias de inventario, garantizando transacciones atómicas en el proceso de checkout, monitoreo automatizado en segundo plano de niveles críticos de stock y auditorías concurrentes de alto rendimiento sin degradar la experiencia de navegación del cliente.

2. Objetivo general.

Desarrollar un Sistema Comercial de Gestión de Inventario con funcionalidades de E-Commerce, basado en una arquitectura monolítica utilizando Java y Spring Boot, que permite la administración centralizada del catálogo de productos, el control de inventario en tiempo real, la gestión de un carrito de compras y la simulación del proceso de pago y del despacho logístico. Asimismo, diseñar e implementar una base de datos relacional en PostgreSQL que garantice la integridad, consistencia y almacenamiento eficiente de la información del sistema.

3. Objetivos específicos.

- Establecer el alcance funcional y los requerimientos del sistema mediante la elaboración de historias de usuario con criterios de aceptación, integrando el diseño e implementación del modelo relacional en PostgreSQL y la configuración de la estructura del repositorio en GitHub para el control de versiones y trabajo colaborativo.

- Desarrollar la lógica de negocio y los módulos principales de la plataforma mediante Spring Boot y JPA/Hibernate, implementando la autenticación de usuarios, la gestión y control de inventario con alertas de stock, y el procesamiento transaccional que abarque la pasarela de pagos y el despacho de ventas.

- Implementar mecanismos avanzados de acceso a datos mediante JDBC puro y configuración optimizada de pool de conexiones, integrando programación concurrente multi-hilo para la auditoría y monitoreo de inventario en tiempo real, garantizando la robustez y calidad del software mediante alta cohesión, bajo acoplamiento, gestión global de excepciones y validación mediante pruebas unitarias.

4. Introducción.

El presente documento detalla la especificación técnica, arquitectónica y de implementación final del Sistema Comercial de Gestión de Inventario con funcionalidades de E-Commerce, desarrollado bajo el ecosistema de Java 21 y Spring Boot. La plataforma integra en una solución unificada dos entornos operativos: una tienda virtual orientada al consumidor final para la exploración de catálogo, gestión de carrito en sesión y compra electrónica, y un panel administrativo diseñado para el control centralizado de inventario, kardex de movimientos, auditoría multihilo, gestión de usuarios/roles y despacho logístico de pedidos. A nivel de infraestructura de datos, el sistema emplea una estrategia de persistencia híbrida sobre PostgreSQL, combinando la agilidad de Spring Data JPA / Hibernate con la eficiencia de bajo nivel de JDBC Puro gestionado mediante un pool de conexiones Tomcat JDBC.

5. SCRUM.

N°	Nombre Completo	Rol Scrum	Rol Técnico
1	Brandon Ismael Mejía Rivera	Product Owner	Analista de Negocio y Requerimientos
2	Vladimir Antonio Ruiz Estrada	Scrum Master	Líder Técnico y Desarrollador Backend
3	Iván Fernando Díaz Escobar	Developer	Desarrollador Frontend
4	Miguel Ángel Serrano Alfaro	Developer	Desarrollador Backend
5	Flamenco Joya, Jairo Joaquin	Developer	Desarrollador Fullstack

6	Alexander Alberto Emestica Ramirez	Developer / DBA	Administrador de Base de Datos
7	Katya Abigail Ruiz Fernández	QA / Tester	Analista de pruebas y Calidad de Software
8	Adrian Enrique Pinto Elizondo	Developer	Desarrollador Backend

6. Definición de roles y funciones por rol del sistema.

Rol del Sistema	Descripción	Funciones en el Sistema
Administrador del Sistema	Encargado del control total del sistema, configuración global y supervisión operativa general.	● Registrar, modificar y desactivar usuarios y empleados. ● Gestionar roles y permisos del sistema. ● Consultar reportes globales de ventas e inventarios. ● Configurar parámetros generales (métodos de pago simulados)
Gestor de Inventario / Almacén	Encargado de la logística interna, catálogo de productos y control físico de existencias.	● Registrar, editar y categorizar productos en el catálogo. ● Registrar entradas y salidas de stock. ● Configurar umbrales de stock mínimo por producto. ● Recibir alertas automáticas de reabastecimiento. ● Actualizar estados de despacho físico de pedidos.
Cliente / Comprador	Usuario final que explora el catálogo de productos y realiza compras en línea.	● Registrarse y gestionar su perfil personal. ● Navegar y filtrar catálogo de productos por categoría y precio. ● Agregar productos al carrito de compras.

		● Procesar orden de compra e ingresar datos de envío y pago. ● Consultar el historial y estado actual de sus pedidos.
Cajero / Procesador de Pedidos	Encargado de la verificación financiera de los pedidos y validación del cobro simulado.	● Consultar pedidos pendientes de validación. ● Confirmar la recepción de pagos simulados. ● Cambiar el estado del pedido a “Listo para Despacho”. ● Generar comprobante electrónico de venta.

7. Requerimientos del sistema.

Prioridad	HU	Criterios
HU-001 (Alta)	Como Gestor de Inventario quiero registrar y catalogar productos con sus detalles (nombre, SKU, precio, stock, marca, categoría, etc) para mantener actualizado el inventario en tiempo real.	● El producto debe quedar almacenado con un código SKU único. ● Validar que el precio y stock sean valores numéricos mayores o iguales a cero. ● Permitir asociar una categoría y marca existente al producto. ● Mostrar confirmación tras el registro exitoso.
HU-002 (Alta)	Como Cliente quiero crear una cuenta de usuario e iniciar sesión para poder guardar mi información de despacho y dar seguimiento a mis compras.	● El correo electrónico debe ser único en el sistema. ● La contraseña debe cifrarse antes de ser guardada en la BD. ● Emitir mensaje de error en caso de

		credenciales incorrectas. ● Permitir recuperación de contraseña básica.
HU-003 (Alta)	Como Cliente quiero agregar productos a un carrito de compras y modificar las cantidades para preparar mi orden de pedido.	● El sistema debe verificar que la cantidad solicitada no supere el stock disponible. ● Calcular automáticamente el total. ● Permitir eliminar ítems o vaciar el carrito.
HU-004 (Alta)	Como Cliente quiero procesar la compra mediante una simulación de pasarela de pago para confirmar la transacción y generar el pedido.	● Validar formato de tarjeta simulada (16 dígitos, fecha exp., CVV). ● Disminuir automáticamente el stock disponible de los productos comprados. ● Cambiar el estado del pedido a “En Proceso/Pagado”. ● Generar un número de orden único.
HU-005 (Media)	Cómo Gestor de Almacén quiero consultar las órdenes listas para despacho para actualizar el estado físico del envío (Pendiente-> Empacado -> Enviado -> Entregado).	● Filtrar pedidos por estado. ● Registrar la fecha y hora de actualización del despacho. ● Notificar el cambio de estado al perfil del cliente.
HU-006 (Media)	Como Gestor de Inventario quiero recibir alertas automáticas cuando el stock de un producto llegue a su nivel mínimo para gestionar el reabastecimiento oportunamente.	● Emitir notificación visual en el panel cuándo Stock Actual < Stock Mínimo. ● Generar lista de productos sugeridos a reabastecer.
HU-007 (Baja)	Como Administrador quiero visualizar reportes estadísticos de productos más vendidos y totales	● Filtrar reportes por rango de fechas. ● Exportar o visualizar métricas de productos más demandados.

8. Alcances y Limitaciones.

Alcances (Lo que SÍ hará el sistema).

- **Gestión de Catálogo e Inventario:** Control CRUD completo de productos, categorías y stock dinámico con alertas de stock bajo.
- **Gestión de Usuarios y Perfiles:** Módulo para clientes y personal interno con control de acceso basados en roles.
- **Carrito de Compras y Flujo de Checkout:** Carrito persistente durante la sesión y cálculo en tiempo real de totales.
- **Simulador de Pasarela de Pago:** Procesamiento simulado de pagos en línea con tarjetas de prueba sin transacción bancaria real.
- **Módulo de Despacho Físico:** Seguimiento de estados logísticos desde la preparación de orden hasta la entrega final simulada.
- **Reportes Básicos:** Generación de resúmenes de inventario y ventas.

Limitaciones y Límites (Lo que NO hará el sistema).

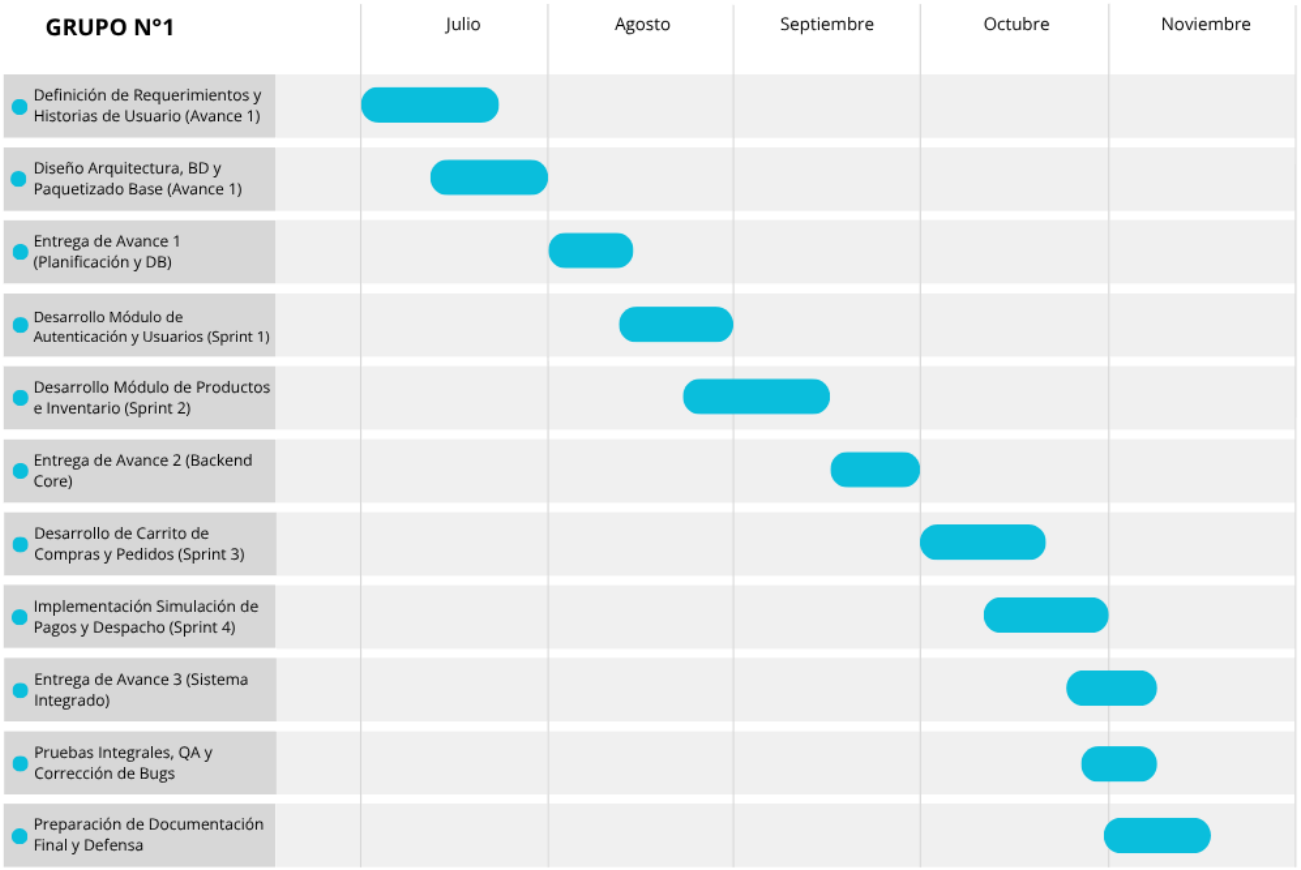
- **NO** realizará cobros reales con bancos ni pasarelas reales en producción, se utilizará un mock/simulador.
- **NO** incluirá integración directa con servicios de mensajería externa mediante API en tiempo real.
- **NO** contará con aplicación móvil nativa (Android/iOS), el alcance se limita a plataforma Web Responsive.
- **NO** implementará arquitectura multi-tenancy o múltiples tiendas independientes.

9. Planificación.

Actividad	Responsables	Fecha de Inicio	Fecha de Fin
Definición de Requerimientos y Historias de Usuario (Avance 1)	Product Owner	16/07/2026	23/07/2026
Diseño Arquitectura, BD y Paquetizado Base (Avance 1)	Scrum Master / DBA	25/07/2026	01/08/2026
Entrega de Avance 1 (Planificación y DB)	Todo el Equipo Scrum	01/08/2026	10/08/2026
Desarrollo Módulo de Autenticación y Usuarios (Sprint 1)	Dev Fullstack / Dev Backend	28/08/2026	07/09/2026
Desarrollo Módulo de Productos e Inventario (Sprint 2)	Dev Backend / Dev Frontend / DBA	06/09/2026	18/09/2026
Entrega de Avance 2 (Backend Core)	Todo el Equipo Scrum	24/08/2026	27/09/2026
Desarrollo de Carrito de Compras y Pedidos (Sprint 3)	Dev Frontend / Dev Backend	07/10/2026	10/10/2026
Implementación Simulación de Pagos y Despacho (Sprint 4)	Dev Fullstack / Dev Backend	12/10/2026	20/10/2026
Entrega de Avance 3 (Sistema Integrado)	Todo el Equipo Scrum	05/10/2026	25/10/2026

Pruebas Integrales, QA y Corrección de Bugs	QA Tester / Devs	23/10/2026	25/10/2026
Preparación de Documentación Final y Defensa	Todo el Equipo Scrum	19/10/2026	26/10/2026

10. Cronograma.



11. Entradas y salidas del sistema.

HU-001: Gestión de Catálogo e Inventario de Productos.

- **Entradas:** Nombre del Producto, código SKU, descripción, precio unitario, stock inicial, stock mínimo, ID de categoría y marca.
- **Salidas:** Confirmación de registro de producto, catálogo actualizado, mensaje de error en caso de SKU duplicado.

HU-002: Registro e Inicio de Sesión de Clientes.

- **Entradas:** Nombre, apellido, correo electrónico, contraseña, teléfono, dirección de envío, credenciales de inicio de sesión (correo y contraseña).
- **Salidas:** Mensaje de bienvenida/registro exitoso, token/sesión de usuario activa, perfil de cliente actualizado, mensaje de error en caso de credenciales inválidas o correo duplicado.

HU-003: Carrito de Compras.

- **Entradas:** ID de producto, cantidad solicitada, ID de cliente en sesión.
- **Salidas:** Vista interactiva del carrito, total general a pagar, notificación de stock insuficiente si aplica.

HU-004: Procesamiento de Pedido y Pasarela Simulada.

- **Entradas:** ID de cliente, dirección de entrega, número de tarjeta simulado, fecha de vencimiento, CVV, lista de ítems del carrito.
- **Salidas:** Número de orden generado (ej: ORD-2026-0001), actualización automática de descuento en stock de BD, cambio de estado a “Pagado”.

HU-005: Control y Despacho Físico.

- **Entradas:** ID de orden, nuevo estado de envío (ej: ENVIADO, ENTREGADO).
- **Salidas:** Notificación de cambio de estado, reporte de órdenes listas para entrega en almacén.

HU-006: Alertas Automáticas de Stock Mínimo.

- **Entradas:** Evento de venta o actualización de inventario, límite mínimo de inventario (stock actual < stock mínimo).
- **Salidas:** Alerta visual destacada en el panel de administración/inventario, reporte de productos en nivel crítico, lista de sugerencia de compra para reabastecimiento.

HU-007: Reportes Estadísticos de Ventas e Inventario.

- **Entradas:** Rango de fechas seleccionadas (fecha inicio, fecha fin), tipo de reporte solicitado (productos más vendidos, ingresos totales).
- **Salidas:** Gráficos y tablas de resumen con métricas clave de ingresos, listado ranking de productos más demandados, cálculo de total acumulado en ventas.

12. Declaración de entidades del sistema.

Esquema: Core

Entidad	Descripción	Atributos
Producto	Artículo comercializable disponible en la tienda.	idProducto (Integer), codigoSku (String), id_categoria (FK - Categoria), id_marca (FK - Marca), nombreProducto (String), descripcionProducto (String), stockDisponible (Integer), stockMinimo (Integer), estado (Character), fechaCreacion (LocalDateTime), metadata (String / JSONB), precioVenta (BigDecimal), precioCosto (BigDecimal)
Pedido	Registro de la orden de compra efectuada por un cliente en la plataforma.	idPedido (Integer), id_usuario (FK - Usuario), id_direccion (FK - UsuarioDireccion), estadoPedido (String), total (BigDecimal), fechaCreacion (OffsetDateTime)
Pago	Transacción financiera asociada a un pedido para liquidar su importe.	idPago (Integer), id_pedido (FK - Pedido / OneToOne), metodoPago (String), monto (BigDecimal), estadoTransaccion (String), fechaPago (OffsetDateTime)

MovimientoInventario	Registro de auditoría sobre entradas, salidas o ajustes en las existencias de un producto.	idMovimiento (Integer), id_producto (FK - Producto), tipoMovimiento (String), cantidad (Integer), motivo (String), fechaMovimiento (OffsetDateTime), id_pedido (FK - Pedido)
Envio	Gestión logística y seguimiento del despacho y entrega de un pedido al cliente.	idEnvio (Integer), id_pedido (FK - Pedido / OneToOne), empresaTransporte (String), numeroSeguimiento (String), estadoEnvio (String), fechaDespacho (OffsetDateTime), fechaEntrega (OffsetDateTime)
DetallePedido	Desglose individual de cada producto, cantidad y subtotales asociados a un pedido específico.	idDetalle (Integer), id_pedido (FK - Pedido), id_producto (FK - Producto), cantidad (Integer), precioUnitario (BigDecimal), subtotal (BigDecimal)
Categoria	Clasificación jerárquica para agrupar y organizar los productos en la tienda.	idCategoria (Integer), id_categoria_padre (FK - Categoria), nombreCategoria (String), descripcion (String), fechaCreacion (LocalDateTime)

Esquema: Catálogo.

Entidad	Descripción	Atributos
Pais	Representación geográfica del país para la localización y cobertura de envíos o direcciones.	idPais (Integer), nombrePais (String)
Departamento	División territorial o estatal asociada a un país para la segmentación geográfica.	idDepartamento (Integer), id_pais (FK - Pais), nombreDepartamento (String)
Municipio	Subdivisión administrativa local perteneciente a un departamento para detallar direcciones de entrega.	idMunicipio (Integer), id_departamento (FK - Departamento), nombreMunicipio (String)
Marca	Identificación comercial o fabricante asociado a los productos del catálogo.	idMarca (Integer), nombreMarca (String), fechaCreacion (LocalDateTime)

Esquema: Security

Entidad	Descripción	Atributos
Usuario	Cuenta principal y credenciales de acceso para la autenticación de usuarios en el sistema, incluyendo mecanismos temporales de restablecimiento de contraseña.	idUsuario (Integer), correoInicioSesion (String), contrasenia (String), tokenReinicio (String), expiracionToken (LocalDateTime), eliminado (Boolean), estado

		(Character), ultimoInicioSesion (LocalDateTime), fechaCreacion (LocalDateTime), metadata (String / JSON)
UsuarioContacto	Información de contacto (teléfono, correo secundario u otros canales) asociada a un usuario.	idContacto (Integer), id_usuario (FK - Usuario), tipoContacto (Integer), valor (String), principal (Boolean), verificado (Boolean)
UsuarioDireccion	Domicilios registrados por el usuario para envíos y facturación, vinculados a su localidad.	idDireccion (Integer), id_usuario (FK - Usuario), id_municipio (FK - Municipio), direccionLinea1 (String), codigoPostal (Integer), tipoDireccion (Character), fechaCreacion (LocalDateTime), metadata (String / JSON)
UsuarioPerfil	Datos personales e identificación legal del usuario dentro de la plataforma.	idPerfil (Integer), id_usuario (FK - Usuario / OneToOne), nombres (String), apellidos (String), dui (String), genero (Character), fechaNacimiento (LocalDate)

13. Proyecto base con la estructura.

```
ecommerce-progra-2/
├── .mvn/
│   └── wrapper/
├── db/
│   ├── e-commerce-design-db.dbml
│   ├── schema-ecommerce.sql
│   ├── seed-data.sql
│   └── migraciones/
├── ecommerce-webapp/
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   ├── java/com/sv/grupo1/ecommerce/
│       │   │   ├── ECommerceApplication.java
│       │   │   ├── controllers/
│       │   │   ├── dao/
│       │   │   ├── dto/
│       │   │   ├── entities/
│       │   │   │   ├── catalogo/
│       │   │   │   ├── core/
│       │   │   │   └── seguridad/
│       │   │   └── services/
│       │   └── resources/
│       │       ├── application.properties
│       │       └── templates/admin/
│       └── test/
├── mock-gateway-api/
│   ├── pom.xml
│   ├── docs/
│   ├── logs/
│   └── src/main/
│       ├── java/com/sv/grupo1/ecommerce/gateway/
```

│ │ ├── config/
│ │ ├── controller/
│ │ ├── dto/
│ │ ├── enums/
│ │ └── service/
│ └── resources/
│ └── application.properties
├── mvnw
├── mvnw.cmd
├── pom.xml
└── README.md

14. Tecnologías.

Java 21
Spring Boot 4.1 (Web, Data JPA, Security, Validation, Thymeleaf)
PostgreSQL
Maven (con wrapper mvnw / mvnw.cmd)

15. Versionamiento del proyecto.

https://github.com/rzvlady/ecommerce-progra-2.git

16. URL del video de avance 2.

Avance 2 Ecommerce

17. POO Aplicada.

La Programación Orientada a Objetos constituye el pilar fundamental del diseño del sistema, estructurado en capas desacopladas mediante los siguientes principios:

17.1. Encapsulamiento.

El encapsulamiento se aplica rigurosamente en todas las entidades, DTOs y componentes de lógica de sesión:

- **Protección de estado**: Todos los campos de las clases (por ejemplo en Producto, Pedido, Usuario y MovimientoInventario) están declarados como private, restringiendo el acceso no autorizado y exponiendo el comportamiento a través de métodos de acceso (getters y setters).

- **Invariantes y reglas de dominio en el objeto:** En CarritoSesion, los elementos del carrito (List<CarritoItem>) se gestionan internamente. Clientes externos no modifican la lista de forma directa, sino que invocan operaciones encapsuladas como agregarItem(...), actualizarCantidad(...) y eliminarItem(...), las cuales recalculan totales, controlan subtotales y garantizan que las cantidades sean consistentes con las reglas de negocio.

- **Seguridad de credenciales**: En la entidad Usuario, el hash de la contraseña (contrasenia) y el estado de borrado lógico (eliminado = false) se encuentran protegidos, evitando exposiciones involuntarias en las capas de presentación.

17.2. Herencia.

La herencia permite compartir atributos comunes y definir jerarquías lógicas:

- **Jerarquía de usuarios del sistema:** La entidad Usuario (en el paquete seguridad) define la estructura base de identidad (idUsuario, correoInicioSesion, contrasenia, perfil, direcciones), de la cual hereda la clase Cliente, permitiendo especializar perfiles y roles sin duplicar definiciones de identidad ni de autenticación.

- **Herencia en el framework de datos:** Todas las interfaces de repositorio heredan de org.springframework.data.jpa.repository.JpaRepository<T, ID>, adquiriendo de forma automática operaciones transaccionales CRUD, ordenamiento y paginación sin necesidad de escribir código repetitivo.

17.3. Polimorfismo.

El polimorfismo se manifiesta tanto en tiempo de compilación como en tiempo de ejecución:

- **Polimorfismo por interfaz:** Los controladores y servicios de aplicación dependen exclusivamente de interfaces y no de clases concretas. Por ejemplo, los controladores interactúan con ProductoService, MovimientoInventarioJdbcDAO o PasarelaPagoService. En tiempo de ejecución, el contenedor de inversión de control (IoC) inyecta las implementaciones correspondientes (ProductoServiceImpl, MovimientoInventarioJdbcDAOImpl, MockPasarelaPagoServiceImpl).

- **Sobreescritura de métodos (@Override):** Se aplica en la implementación de contratos como loadUserByUsername(String username) dentro de UserDetailsServiceImpl (que implementa UserDetailsService de Spring Security), adaptando la resolución polimórfica de credenciales desde la base de datos PostgreSQL hacia el contexto de seguridad del servidor.

17.4. Abstracción.

- **Se aíslan los detalles complejos de la infraestructura frente a la lógica operativa. La pasarela de pagos se abstrae a través del contrato PasarelaPagoService:** el flujo de compra (CheckoutService) delega la autorización de cobros a través de un DTO estandarizado (SolicitudPagoPasarelaDTO), ignorando si el procesamiento se realiza mediante un mock local HTTP o una entidad bancaria real externa.

- A nivel de persistencia, la gestión física del socket TCP y el protocolo de red hacia la base de datos PostgreSQL se abstraen mediante la interfaz estándar javax.sql.DataSource**.**

17.5. Interfaces.

El sistema define contratos formales para garantizar la extensibilidad y desacoplamiento:

- **MovimientoInventarioJdbcDAO:** Contrato para las operaciones atómicas de auditoría y kardex mediante JDBC puro (insertar, buscarPorId, listarTodos, listarPorProducto, actualizarMotivoYCantidad, eliminar, contarPorTipo).

- **AuditoriaInventarioConcurrenteService:** Expone el contrato de auditoría masiva paralela ejecutarAuditoriaConcurrente(int hilosConcurrencia).

- **StockMonitorTask:** Contrato para tareas en segundo plano programadas de monitoreo de inventario.

- **Interfaces de biblioteca estándar de Java utilizadas:** Callable<V>, Future<V>, Runnable, ThreadFactory, Connection, PreparedStatement, ResultSet.

18. Colecciones y Genéricos.

El proyecto hace un uso exhaustivo del framework de colecciones y tipos genéricos de Java para garantizar la seguridad tipográfica en tiempo de compilación (*type-safety*) y un manejo óptimo de memoria:

18.1. Tipos Genéricos (*Generics*).

- **Persistencia Tipada:** Los repositorios de datos aprovechan la genericidad de Spring Data mediante la parametrización de tipo de entidad y tipo de clave primaria: JpaRepository<Producto, Integer>, JpaRepository<Usuario, Integer>, JpaRepository<Pedido, Integer>, lo que evita el uso de conversiones inseguras (casting).

- **Contenedores Opcionales:** Se implementa intensivamente Optional<T> (como Optional<Producto> u Optional<MovimientoInventario>) para representar de forma explícita la presencia o ausencia de registros de base de datos, eliminando riesgos de excepciones NullPointerException.

- **Genericidad en Programación Asíncrona**: En el módulo concurrente se utilizan tipos genéricos de concurrencia: Callable<AuditoriaCategoriaItem>, Future<AuditoriaCategoriaItem> y List<Future<AuditoriaCategoriaItem>>, garantizando que el recolector de tareas devuelva exactamente el DTO correspondiente a cada hilo sin ambigüedades.

18.2. Estructuras de Colección.

- **ArrayList<E>:** Utilizada para la acumulación dinámica en lecturas JDBC (en MovimientoInventarioJdbcDAOImpl.listarTodos() y listarPorProducto()) y para ensamblar el lote de tareas concurrentes a delegar al pool de ejecución.

- **List.of() y List.copyOf() (Inmutabilidad):** Implementado en StockMonitorTaskImpl para almacenar la lista de productos alertados. Al usar List.copyOf(...), se crea una instantánea inmutable que puede ser leída de manera concurrente y segura por múltiples hilos HTTP sin peligro de mutación concurrente (ConcurrentModificationException).

- **Map<String, Object>:** Utilizado para la propiedad metadata de la entidad Usuario, permitiendo persistir atributos no estructurados mapeados contra columnas tipo JSONB de PostgreSQL.

19. Programación Concurrente.

19.1. Qué proceso se ejecuta concurrentemente.

El sistema implementa dos procesos concurrentes diferenciados:

1. Auditoría Masiva de Inventario por Categorías (AuditoriaInventarioConcurrenteServiceImpl): Divide el catálogo completo en categorías y asigna la verificación de existencias y balance de movimientos de **inventario de cada categoría a un hilo de trabajo independiente de forma paralela.**

2. **Monitoreo Continuo de Niveles Críticos de Stock (StockMonitorTaskImpl):** Proceso ejecutado en segundo plano cada 60 segundos que inspecciona productos cuyo stock disponible sea menor o igual al stock mínimo configurado.

19.2. Por qué se utiliza concurrencia

- **Reducción del tiempo de respuesta:** La verificación secuencial del balance de inventario y movimientos kardex para miles de registros provocaría tiempos de espera prolongados en la interfaz web administrativa. La concurrencia aprovecha la arquitectura multinúcleo del procesador para ejecutar la auditoría de múltiples categorías en paralelo.

- **No bloqueo de la experiencia de usuario:** Las tareas de monitoreo de stock se desacoplan de los hilos de atención HTTP de Tomcat, evitando que las solicitudes de los clientes en la tienda sufran demoras por tareas pesadas de mantenimiento.

19.3. Mecanismo utilizado.

- **ExecutorService con Pool de Hilos Fijo**: Se utiliza Executors.newFixedThreadPool(poolSize, threadFactory), donde el tamaño del pool se ajusta dinámicamente según la carga solicitada (hasta 10 hilos simultáneos).

- **ThreadFactory Personalizado:** Se implementa una factoría de hilos con nombres legibles (audit-worker-thread-X) usando AtomicInteger para facilitar la trazabilidad en logs, configurándolos además como hilos demonio (setDaemon(true)).

- **Callable<T> y Future<T>**: Las subtareas de auditoría se encapsulan como objetos Callable<AuditoriaCategoriaItem>, permitiendo recolectar los resultados mediante executor.invokeAll(tareas, 60, TimeUnit.SECONDS), controlando tiempos máximos de espera mediante timeout.

- **Control de Memoria y Visibilidad (volatile)**: En StockMonitorTaskImpl, el recurso compartido que contiene los productos en alerta crítica se declara como private volatile

List<Producto> productosBajoStockAlertados. La palabra clave volatile asegura que las escrituras del hilo del monitor sean visibles de inmediato e incondicionalmente para los hilos de peticiones web que leen el estado desde el panel de control.

- **Apagado Seguro (Graceful Shutdown)**: Uso de shutdown() y awaitTermination() en bloques finally para evitar fugas de subprocesos al culminar la tarea.

19.4. Cómo interactúa con la Base de Datos.

- **Aislamiento absoluto por hilo:** Cada hilo trabajador solicita su propia conexión a PostgreSQL a través del pool administrado (dataSource.getConnection()) dentro de una sentencia try-with-resources.

- **Cero contención en JDBC:** Ningún hilo comparte objetos Connection, PreparedStatement ni ResultSet. Cada hilo ejecuta su consulta de agregación (SELECT ... GROUP BY), computa los totales de su categoría y cierra/devuelve la conexión al pool de inmediato.

- Las consultas son de solo lectura, minimizando la retención de bloqueos a nivel de tablas en la base de datos relacional.

20. JDBC.

Para operaciones críticas de auditoría y registro de movimientos de inventario se implementó acceso a datos mediante JDBC Puro, desacoplado del ORM:

20.1. Conexión.

La conexión se obtiene directamente del DataSource inyectado mediante dataSource.getConnection(). Se utiliza siempre dentro de la estructura try-with-resources de Java, asegurando que ante cualquier éxito o error la conexión sea retornada al pool de forma automática.

20.2. DataSource.

Se implementa la interfaz estándar javax.sql.DataSource, concretada mediante la biblioteca Tomcat JDBC Pool (org.apache.tomcat.jdbc.pool.DataSource), centralizando la URL, credenciales de conexión y parámetros de rendimiento.

20.3. Pool de Conexiones.

El pool mantiene un conjunto de conexiones físicas pre-establecidas hacia el servidor de base de datos PostgreSQL alojado en la nube. Cuando un componente requiere ejecutar una consulta, toma una conexión libre del pool en microsegundos, evitando la penalización de rendimiento del protocolo de enlace TCP (*handshake*) y la negociación SSL repetida en cada petición.

20.4. Patrón DAO / Repository-

Se adopta el patrón de diseño Data Access Object:

- Interfaz: MovimientoInventarioJdbcDAO define los métodos del modelo sin ataduras tecnológicas.
- Implementación: MovimientoInventarioJdbcDAOImpl (anotada con @Repository) encapsula todas las instrucciones SQL nativas, control de transacciones y mapeo de datos, impidiendo que la lógica de negocio se contamine con detalles SQL.

20.5. PreparedStatement.

Todas las operaciones JDBC se ejecutan mediante `PreparedStatement`:

- Seguridad: Los parámetros se envían mediante marcadores posicionales (?), como ps.setInt(1, idProducto) y ps.setString(2, motivo), garantizando inmunidad total contra ataques de Inyección SQL (SQLi).

- Rendimiento: Permite al motor PostgreSQL reutilizar los planes de ejecución precompilados de las consultas.

20.6. ResultSet.

El ResultSet actúa como cursor tabular sobre las tuplas retornadas por la consulta. Se procesa mediante iteraciones while (rs.next()) o if (rs.next()). La transformación a objetos de dominio se centraliza en el método privado mapearMovimiento(ResultSet rs), controlando la conversión de tipos (getInt, getString, getTimestamp) y validando valores nulos en columnas relacionales mediante rs.wasNull() para campos opcionales como id_pedido.

21. Configuración del DataSource.

El pool de conexiones se configuró programáticamente en la clase @Configuration DataSourceConfig.java utilizando Tomcat JDBC Pool. A continuación se documentan los parámetros técnicos implementados y su justificación funcional:

Parámetro	Valor Configurado	Justificación y Función
driverClassName	**org.postgresql.Driver**	**Driver JDBC oficial para la comunicación con el motor PostgreSQL.**
url	**jdbc:postgresql://...:5432/postgres?sslmode=require**	**Cadena de conexión hacia el clúster de base de datos con cifrado TLS/SSL forzado.**

initialSize	**5**	**Número de conexiones físicas que se crean inmediatamente al arrancar la aplicación, eliminando latencia en las primeras peticiones.**
minIdle	**5**	**Conexiones mínimas que el pool mantiene siempre abiertas y listas en estado ocioso para atender picos repentinos de tráfico.**
maxIdle	**10**	**Límite máximo de conexiones ociosas que pueden permanecer en reposo antes de que el pool comience a liberarlas.**

maxActive	**20**	**Capacidad máxima de conexiones concurrentes simultáneas hacia la base de datos para evitar sobrecargar los recursos del servidor PostgreSQL.**
maxWait	**10000 (10 segundos)**	**Tiempo máximo en milisegundos que un hilo esperará por una conexión disponible antes de lanzar una excepción de timeout.**
validationQuery	**SELECT 1**	**Sentencia SQL ligera ejecutada para confirmar que la conexión física con el host remoto sigue activa antes de utilizarla.**

testOnBorrow	**true**	**Obliga a validar la conexión antes de entregarla al hilo de la aplicación, previniendo errores de sockets caídos.**
testOnConnect	**true**	**Verifica la viabilidad de la conexión inmediatamente después de haber sido creada por el pool.**
testWhileIdle	**true**	**Un hilo de fondo valida periódicamente las conexiones ociosas en el pool para detectar desconexiones remotas.**

testOnReturn	**false**	**Optimización que omite la validación al regresar la conexión al pool, ahorrando una consulta SQL innecesaria por operación.**
timeBetweenEvictionRunsMillis	**30000 (30 segundos)**	**Intervalo de ejecución del hilo recolector que inspecciona conexiones ociosas o caducadas.**
minEvictableIdleTimeMillis	**60000 (60 segundos)**	**Tiempo mínimo que una conexión debe permanecer ociosa antes de ser candidata a desalojo o reciclaje.**

removeAbandoned	**true**	**Mecanismo de seguridad contra fugas de conexiones (connection leaks): recupera conexiones abiertas que hilos abandonaron sin cerrar.**
removeAbandonedTimeout	**60 (60 segundos)**	**Tiempo de uso continuo a partir del cual el pool califica una conexión como abandonada y fuerza su cierre/recuperación.**
validationInterval	**30000 (30 segundos)**	**Evita revalidar la misma conexión más de una vez dentro de esta ventana de tiempo, mitigando sobrecoste en el servidor.**

21. Cohesión y Acoplamiento.

21.1. Alta Cohesión.

Cada clase y paquete dentro de la aplicación cumple una única responsabilidad bien delimitada (Principio de Responsabilidad Única):

- MovimientoInventarioJdbcDAOImpl tiene como único propósito ejecutar el CRUD relacional en la tabla core.movimientos_inventario. No valida reglas de negocio de pedidos ni formatea respuestas HTML.

- AuditoriaInventarioConcurrenteServiceImpl se encarga exclusivamente de la partición de tareas, creación del pool de hilos y cálculo del balance de stock por categoría.

- StockMonitorTaskImpl se limita a la vigilancia periódica de niveles críticos de stock y la notificación a logs.

- Los objetos de transferencia de datos (DTOs) únicamente encapsulan y transportan información entre capas, sin contener lógica de negocio ni dependencias del framework de persistencia.

21.2. Bajo Acoplamiento.

- Inversión de Control (IoC) e Inyección de Dependencias (DI): Los controladores y servicios nunca instancian directamente a sus colaboradores (new ServicioImpl()). Todas las dependencias se inyectan a través del constructor, desacoplando las clases de implementaciones concretas.

- Programación orientada a interfaces: Los servicios dependen de contratos abstractos (ProductoRepository, MovimientoInventarioJdbcDAO, PasarelaPagoService), permitiendo cambiar la implementación subyacente (por ejemplo, reemplazar el mock de pagos por un proveedor real) sin modificar ni una sola línea de la lógica de negocio de los pedidos.

22. Legibilidad y Reutilización.

- **Convenciones de nomenclatura:** Se siguen las convenciones de Java (notación CamelCase para clases, camelCase para métodos/atributos y UPPER_SNAKE_CASE para constantes como las definidas en Constantes.java).

- **Modularidad de vistas con Thymeleaf:** Se construyeron plantillas reutilizables mediante fragmentos (fragments/head.html, fragments/ecommerce-navbar.html, fragments/topbar.html, fragments/sidebar.html) integrados en layouts maestros (admin.html y ecommerce-layout.html), erradicando la duplicación de código en la capa de presentación.

- **Reutilización de componentes transversales:** Los DTOs se diseñaron con propósito específico (ProductoHomeDTO, ItemCompraDTO, CheckoutDTO) para ser consumidos tanto por vistas web tradicionales como por controladores REST (BusquedaApiController, CarritoApiController).

- **Código autodocumentado:** El código cuenta con comentarios Javadoc en interfaces y métodos principales, acompañados de trazas de registro estructuradas (logs parametrizados con SLF4J) que facilitan la depuración sin introducir sobrecarga de procesamiento de cadenas**.**

23. Manejo de Excepciones.

El sistema cuenta con una arquitectura de captura y traducción de fallos robusta, centralizada y orientada a la seguridad:

[Capa de Persistencia JDBC / JPA]

|

Lanza SQLException / DataAccessException

|

▼

[Traducción en Capa DAO / Service]

│

Lanza DatabasePersistenceException (Excepción personalizada no chequeada)

│

▼

[Interceptor Central @ControllerAdvice: GlobalExceptionHandler]

│ 1. Loguea causa técnica en archivo de logs (SLF4J) │

│ 2. Asigna código de estado HTTP adecuado (503, 401, 500) │

│ 3. Redirige a vista 'error.html' con mensaje amigable │

23.1. Jerarquía de Excepciones Propias.

- **DatabasePersistenceException:** Excepción personalizada en tiempo de ejecución (RuntimeException) que encapsula fallos en operaciones JDBC puro, preservando el código de estado SQL (sqlState) y el código de error del motor sin exponer detalles técnicos al usuario.

- **CredencialesInvalidasException:** Señaliza intentos fallidos de autenticación por credenciales incorrectas.

23.2. Manejador Global Centralizado (@ControllerAdvice).

La clase GlobalExceptionHandler intercepta todas las anomalías que escapan de los controladores web y las mapea a respuestas semánticas:

- **Fallas de base de datos (DatabasePersistenceException, SQLException, DataAccessException):** Responde con código HTTP 503 SERVICE UNAVAILABLE. Se muestra la vista amigable error.html notificando indisponibilidad temporal del servicio, impidiendo la fuga de información sensible (como nombres de tablas o cadenas de conexión).

- **Fallas de autenticación (CredencialesInvalidasException):** Responde con HTTP 401 UNAUTHORIZED.

- **Recursos no encontrados (NoResourceFoundException):** Responde con HTTP 404 NOT FOUND.

- **Errores no controlados (Exception):** Captura general que responde con HTTP 500 INTERNAL SERVER ERROR, registrando la traza completa de error (stacktrace) en el archivo de log (e-commerce-web-application-log.log) mediante log.error(...).

24. Pruebas Unitarias.

La verificación de calidad del software se llevó a cabo utilizando JUnit 5, Mockito y el plugin de cobertura JaCoCo, asegurando el correcto aislamiento de la lógica de negocio frente a la infraestructura externa.

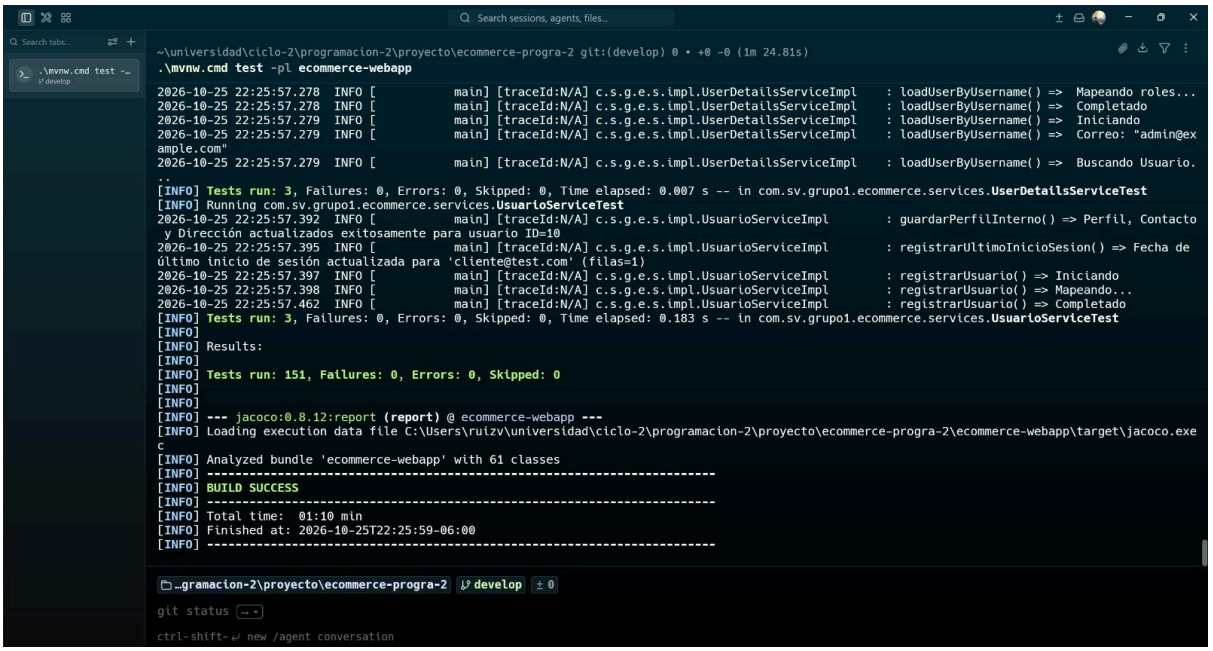
Clase de Prueba	Componente Evaluado	Tipo de Prueba	Escenario Validado

MovimientoInventarioJdbcDAOTest	**MovimientoInventarioJdbcDAOImpl**	**Persistencia JDBC**	**Inserción exitosa de movimiento, recuperación de llaves generadas, mapeo correcto de ResultSet y captura de DatabasePersistenceException ante fallas SQL simuladas.**
AuditoriaInventarioConcurrenteServiceTest	**AuditoriaInventarioConcurrenteServiceImpl**	**Concurrencia**	**Ejecución paralela de tareas de auditoría utilizando múltiples hilos, agregación de métricas y cumplimiento de tiempos de respuesta.**
StockMonitorTaskTest	**StockMonitorTaskImpl**	**Segundo Plano / Task**	**Detección periódica de productos por debajo del umbral mínimo y actualización segura en memoria (volatile).**

BackofficeInventarioServiceTest	**BackofficeInventarioServiceImpl**	**Lógica de Negocio**	**Ajustes de stock, validación de cantidades positivas y registro de movimientos asociados.**
CheckoutServiceTest	**CheckoutServiceImpl**	**Transaccional**	**Proceso completo de confirmación de pedido, decremento de existencias y enlace con simulación de pago.**
MockPasarelaPagoServiceImplTest	**MockPasarelaPagoServiceImpl**	**Integración Mock**	**Autorización y rechazo de transacciones según montos y datos de tarjeta de crédito simulados.**
DatabasePersistenceExceptionTest	**DatabasePersistenceException**	**Manejo de Errores**	**Verificación de propagación de mensaje, causa y metadatos de estado SQL.**

CarritoSesionServiceTest	**CarritoSesion**	**Lógica de Dominio**	**Adición, actualización y eliminación de artículos en sesión de compras, comprobando recálculo de importes.**

25. Evidencias de ejecución.



26. Git / GitHub.

El desarrollo del proyecto se gestionó bajo buenas prácticas de control de versiones distribuido utilizando Git y la plataforma GitHub:

- **Estrategia de Ramificación:** Se implementó un flujo basado en ramas funcionales:
  - **main / master:** Contiene únicamente versiones estables y listas para despliegue.
  - **develop:** Rama integradora de los diferentes módulos del sistema.

- **Ramas de características (feature/*):** Cada módulo o requerimiento (como feature/concurrencia-inventario, feature/jdbc-dao, feature/pool-datasource) se desarrolló en una rama aislada antes de ser integrado.

- **Buenas Prácticas de Commits:** Se utilizaron mensajes semánticos y atómicos siguiendo convenciones estandarizadas:
  - feat: para incorporación de nuevas funcionalidades.
  - fix: para corrección de bugs o ajustes de validación.
  - refactor: para mejoras en cohesión y desacoplamiento de clases.
  - test: para adición y configuración de la suite de pruebas unitarias.
  - docs: para actualizaciones en documentación técnica del proyecto.

27. Conclusiones.

1. **Eficiencia en la Persistencia de Datos:** La coexistencia de Spring Data JPA para la gestión del modelo de entidades de negocio junto con JDBC Puro para transacciones específicas de auditoría e inventario demostró ser una solución arquitectónica equilibrada, combinando la alta productividad de un ORM moderno con la velocidad y control granular que ofrece JDBC directo.

2. **Optimización de Recursos Mediante Pools de Conexiones**: La configuración afinada de Tomcat JDBC Pool (controlando tamaños mínimos, máximos, tiempos de espera y mecanismos de reciclaje de conexiones abandonadas) garantiza que el sistema soporte picos de concurrencia comercial sin degradar el rendimiento ni sobrecargar la base de datos relacional PostgreSQL.

3. **Escalabilidad a Través de Programación Concurrente:** El uso coordinado de ExecutorService, Callable, Future y tareas programadas (@Scheduled) permite ejecutar auditorías complejas y monitoreo de inventario de forma asíncrona, eliminando cuellos de botella y manteniendo tiempos de respuesta óptimos para los usuarios de la plataforma de comercio electrónico.

4. **Mantenibilidad y Calidad de Código:** La aplicación estricta de principios de orientación a objetos (alta cohesión, bajo acoplamiento e inversión de dependencias), complementada con una política centralizada de manejo de excepciones y una batería de pruebas unitarias automatizadas con cobertura JaCoCo, consolidan un producto de software robusto, mantenible y alineado a los estándares de la industria.

5. **Alineación Estratégica con Metodología Scrum:** La organización del equipo en 7 roles definidos permite una clara separación de responsabilidades, garantizando una comunicación fluida entre el área funcional (Product Owner) y la ejecución técnica (Scrum Master, Developers, QA).

6. **Definición Clara del Alcance e Inventario:** Se han establecido límites precisos en las historias de usuario para acotar el desarrollo a un simulador de e-commerce robusto enfocado en la consistencia de inventarios en tiempo real y la gestión del flujo de órdenes.

7. **Arquitectura de Software Escalable:** La estructura de paquetes en Java (model, dao, service, repository) y las entidades modeladas establecen una base sólida para el desarrollo modular de los siguientes avances hacia la defensa final.

8. **Modularidad y Mapeo Multiesquema:** La implementación del modelo relacional mediante JPA/Hibernate se organizó eficazmente en subpaquetes (catalogo, core, seguridad), lo que asegura una correspondencia directa con la base de datos PostgreSQL y facilita el mantenimiento desacoplado entre las parametrizaciones generales, las reglas transaccionales y el módulo de usuarios.

28. Bibliografía.

- Schwaber, K., & Sutherland, J. (2020). *La Guía de Scrum: Las Reglas del Juego.* Scrum.org.

- Oracle. (2024). *Serializable Objects.* The Java Tutorials. https://docs.oracle.com/javase/tutorial/jndi/objects/serial.html